哈爾濱工程大學

第 11 讲 / 全 17 讲

C 语言递归用法详解

自己调用自己 —— 用一层层 " 缩小 " 解决大问题

递归思想

从套娃与查字典说起，什么是自己调用自己

基准情形

出口 + 逼近出口，递归成立的两个必要条件

调用栈

每层调用一份副本，先展开、后回收

经典问题

阶乘、斐波那契、gcd 、数位和、进制转换

主讲教师：郎大鹏

本节目标

哈爾濱工程大學

1. 用套娃、查字典等生活例子，说清 " 自己调用自己 " 的递归思想
2. 会把阶乘写成递归定义： $n! = n \times (n-1)!$ ，且 $0! = 1$
3. 记住递归的两个必要条件：基准情形 + 向基准逼近
4. 会写第一个递归函数 fact(n) ，并追踪 fact(4) 的展开与回收过程
5. 能用递归解决斐波那契、 gcd 、数位求和、进制转换、汉诺塔
6. 认识递归的六类常见错误，学会判断什么时候该用递归

01

递归思想

从俄罗斯套娃与查字典说起：什么是 " 自己调用自己 "

哈爾濱工程大學

生活中的递归：套娃与查字典

递归，就是在解决问题的过程中，又用到了 " 解决同一个小一号问题 " 的办法。生活里你早就见过它：打开一个大问题，里面站着一个一模一样、只是更小的自己。

例子一

俄罗斯套娃

打开一个娃娃，里面是一个同款但更小的娃娃；再打开，还有更小的……

有 " 底 " ：最小的实心娃娃，打不开为止

例子二

查字典

查一个词，解释里又有不认识的词，接着查下去……

有 " 底 " ：遇到认识的词就停

反例

两面镜子对照

镜子里有镜子里的镜子，影像一层套一层，无穷无尽……

没有 " 底 " ：程序里这样写就停不下来

一句话 递归 = 同样的办法 + 更小的规模 + 一个停下来的 " 底 "

阶乘的递归定义：5! = 5 × 4!

把 5 × 4 × 3 × 2 × 1 从中间拆开看看：

```
5! = 5 × (4 × 3 × 2 × 1) = 5 × 4!

4! = 4 × (3 × 2 × 1)     = 4 × 3!

3! = 3 × (2 × 1)         = 3 × 2!

2! = 2 × (1)             = 2 × 1!
```

1! = 1 × 0!　　而　　0! = 1　← 到底了

每一行都是同一个形状：自己 × 更小一号的自己

递归定义

`n! = n × (n-1)!`

规模缩小一步，形状完全一样

基准情形

`0! = 1`

不用再拆，直接给出答案的那一行

合起来

一个定义引用了自己，另一个定义兜底 ——

这就是递归的全部秘密

把问题交给 " 缩小版的自己 "

哈尔滨工程大学

要算 5! ，我不用从头乘到尾。我只需要做一件小事：拿出一个 5 乘上去；剩下的 4! 怎么算？——交给 " 会算 4! 的那个人 " 。而那个人，恰好也是我，只是手里的任务更小。

大问题

算 5!

我只做一步：乘上一个 5

→

缩小一号

剩 4!

原样交给 " 小一号的我 " 接着办

→

最小的我

0! = 1

这个我能直接报答案，不用再交出去

关键直觉 每一层只干一小步，然后把 " 同一类的小问题 " 原样外包；答案沿着原路一层层乘回来

递归的两个必要条件

哈尔滨工程大学

条件一 · 基准情形（递归的出口）

必须存在某种足够小的情形，不用再递归就能直接给出答案。

例：`n == 0` 时，直接 `return 1;`（0! = 1）

条件二 · 向基准逼近（每次都要变小）

每一次递归调用，问题规模必须比上一层更小，一步步靠近出口。

例：算 `fact(n)`，先算 `fact(n - 1)`，`n` 一层层减，总会到 `0`

缺一不可 没有条件一：永远停不下来；没有条件二：出口永远等不到 —— 两者都崩

第一个递归函数 fact(n)

```c
int fact(int n) {    // 任务：返回 n 的阶乘
    if (n == 0) {
        return 1;
    }                    // 基准情形：出口
    return n * fact(n - 1);  // 递归调用
}
// 缩小一步：n-1，保证最终到达出口
int main(void) {
    int n;
    scanf("%d", &n);
    printf("%d", fact(n)); // 输入 5 → 输出 120
    return 0;
}
```

① 函数在调用自己

fact 的函数体里出现了 fact(n - 1) —— 这一行就是递归调用，语法上和普通调用没区别

② 参数在变小

传下去的是 n - 1，不是 n，所以一层比一层小，保证能到达 n == 0

③ 出口先写好

n == 0 直接返回 1，不再调用自己 —— 这一行是整个函数的安全绳

递归 vs 循环：两种算阶乘的直觉

哈尔滨工程大学

循环：自己一步步推

```
int s = 1;
int i;
for (i = 1; i <= n; i++) {
    s = s * i;
}
```

像一个人搬砖：从 1 开始，一个数一个数乘上去，自底向上，全程自己动手

递归：只做一步再外包

```
if (n == 0) {
    return 1;
}
return n * fact(n - 1);
```

像当包工头：只乘一个 n ，剩下的活转包给 " 小一号的自己 " ，自顶向下等答案

对比 结果完全一样；递归代码更贴近数学定义，读起来就是定义本身；循环更直接、没有层层调用的开销。会判断 " 这个问题适不适合外包 " ，比背写法更重要

02

递归执行过程

递归调用到底是怎么一步步跑完的：展开、调用栈、常见翻车

哈爾濱工程大学

fact(4) 完整追踪：先展开，后回收

哈爾濱工程大學

展开：一层层问下去 ↓

```
fact(4)  →  4 × fact(3)
fact(3)  →  3 × fact(2)
fact(2)  →  2 × fact(1)
fact(1)  →  1 × fact(0)
fact(0)  →  直接返回 1（基准）
```

回收：带着答案算回来 ↑

```
fact(0) = 1 （基准给出答案）
fact(1) = 1 × 1 = 1
fact(2) = 2 × 1 = 2
fact(3) = 3 × 2 = 6
fact(4) = 4 × 6 = 24 （最终答案）
```

口诀 先展开、后回收：先一路问到底碰到基准，再一层层把答案乘回去

调用栈：每层一份自己的 n

① 每次调用 = 一份新副本

调用 fact(4) 、 fact(3) 、 fact(2)…… 每一次调用都有自己的 n 。呼应函数课：传参就是复制，值传递

② 层与层互不干扰

里层怎么折腾自己的 n ，也动不了外层的 n ；一层算完返回，它那份副本就撤掉

③ 栈：先叠后撤

展开 = 一层层叠上去；回收 = 一层层撤下来，最后回到 main 。叠的层数太多就会撑爆（下一页讲）

fact(4) 展开时的栈（越往下越先入栈）

```
fact(0)    自己的 n = 0
fact(1)    自己的 n = 1
fact(2)    自己的 n = 2
fact(3)    自己的 n = 3
fact(4)    自己的 n = 4
main       自己的 n = 4
```

六个 n ，六份独立副本，谁也不覆盖谁

没有基准情形，会发生什么


```
int bad(int n) {    //  忘了写基准情形
    return n * bad(n - 1);
    }                   //  一直往下调用
}
int main(void) {
    printf("%d", bad(5));
    // bad(5)→bad(4)→bad(3)→… 没有尽头
}
```

后果 屏幕上什么也不输出，程序崩溃退出

真相 每层调用都占一份栈内存，无休止叠下去，栈被耗尽：栈溢出（ Segmentation fault ）

① 它在找什么？

bad 想等到一个 " 不用再递归 " 的情形，可函数里根本没有这样的分支 —— 只能一直等下去

② n 一路变负

bad(5) 、 bad(4) 、……、 bad(0) 、 bad(-1) 、 bad(-2)…… 没有任何一行说 " 到我了，停下 "

③ 正确写法

在递归调用之前先写好： if (n == 0) return 1;

—— 出口必须存在，而且必须写

基准放错位置：够不着的出口

```
int fact(int n) {      //  想求阶乘
    if (n == 1) {
        return 1;
    }                  //  基准写成了  n == 1
    return n * fact(n - 1);
}
int main(void) {
    printf("%d", fact(0));
    // 0!  明明应该是  1，结果呢?
}
```

运行 fact(0) 就崩了

n == 1 不成立 → 调 fact(-1) 、 fact(-2)..... 出口永远够不着

① 错在哪

0! = 1 才是真正的基准，出口却写在 n == 1 上；n 从 0 出发只会一路变负，永远碰不到 1

② 怎么修

把基准改回 n == 0 ；更稳的写法是 n <= 1 ，把 0 和 1 一起兜住

③ 教训

出口不仅要写，还要 " 进得去 " ：从最初的输入出发，一格一格模拟，确认真的能踩到它

递归三要素总结

哈爾濱工程大學

1 **先说清任务：这个函数返回什么**

一句话说死，如： fact(n) 返回 n 的阶乘 —— 任务不明，递归没法写

2 **找到基准情形：什么规模直接给答案**

如： n == 0 时 return 1; —— 出口要存在、要可达（第 13 、 14 页的两种崩法）

3 **写缩小的一步：怎么用小一号的答案**

如： n × fact(n - 1) —— 每层只做一小步，并保证规模真的在变小

口诀：任务 → 出口 → 缩小一步，三样齐了，递归就成型了

什么时候该用递归

判断口诀：这个问题能不能拆成 " 同类型的、小一号的问题 " ？能拆，就考虑递归；不能拆，老老实实用循环。看几个典型场景：

适合

定义本身是递归的

阶乘 n! = n × (n-1)! ；斐波那契：每一项 = 前两项之和。定义长什么样，代码就长什么样

适合

一层层剥开的问题

汉诺塔、十进制转二进制：处理当前一层，剩下的交给小一号的同款问题

不适合

简单重复运算

只是从 1 加到 n 、打印十次——用 for 更直接，绕一圈递归纯属多余

注意

递归有开销

每层调用都要占一份栈内存；层次太深（比如几万层）会把栈撑爆，程序崩溃

一句话 问题天然分层时优先想递归；递归是 " 清晰 " 的工具，不是 " 炫技 " 的工具

03

经典递归问题

五个必会例子：斐波那契 · gcd · 数位求和 · 进制转换 · 汉诺塔

哈爾濱工程大學

斐波那契数列：前两项之和

```c
int fib(int n) {    // 第 n 个斐波那契数
    if (n <= 2) {
        return 1;
    }               // 基准：前两项都是 1
    return fib(n - 1) + fib(n - 2);
}
int main(void) {
    printf("%d", fib(6));
    // 数列 1 1 2 3 5 8，输出 8
}
```

和 fact 最大的不同：分叉

fib(6) = fib(5) + fib(4) ：一次调用变两次，展开是一棵树

① 数列长什么样

1 1 2 3 5 8 13 …

从第 3 项起，每一项 = 前两项之和

② 两个基准

fib(1) 和 fib(2) 都是 1 —— 用 n <= 2 一行兜住两个出口

③ 代码即定义

fib(n) = fib(n-1) + fib(n-2) ，数学定义原样翻译成一行代码

递归求 gcd ：辗转相除的递归版


```c
int gcd(int a, int b) {
    if (b == 0) {
        return a;
    }           // 基准：除数为 0
    return gcd(b, a % b);   // 余数接棒
}
int main(void) {
    printf("%d", gcd(24, 36));
    // 输出 12
}
```

循环版（ while 课写过 ）

```c
while (b != 0) {
    t = a % b;
    a = b;
    b = t;
}
```

追踪 gcd(24, 36)

gcd(24,36) → gcd(36,24) → gcd(24,12) → gcd(12,0) → 12

递归版一句顶四行

gcd(a, b) = gcd(b, a % b) ，这就是辗转相除定义本身； b == 0 时 a 就是最大公约数

求各位数字之和： sumDigits

```
int sumDigits(int n) {
    if (n < 10) {
        return n;
    }           //  基准：只剩一位数
    return n % 10 + sumDigits(n / 10);
}
int main(void) {
    printf("%d", sumDigits(1234));
    //  输出  1+2+3+4 = 10
}
```

追踪 sumDigits(1234)

1234 → 4 + sumDigits(123) → 4+3+2+1 = 10

① 拆位两件套

n % 10 取出末位， n / 10 砍掉末位 —— 纯算术就能逐位拆解，不需要任何新工具

② 基准：只剩一位

n < 10 时它自己就是和，直接返回 n ；用 < 而不是 == ，负数输入也兜得住

③ 天然分层

1234 → 123 → 12 → 1 ，规模每次砍掉一位，问题越变越小，正是递归喜欢的形状

十进制转二进制：递归输出

```
void toBin(int n) {    //  递归输出二进制
    if (n > 1) {    //  先递归处理更高位
        toBin(n / 2);
    }
    printf("%d", n % 2); //  回来再打印当前位
}
int main(void) {
    toBin(13);           //  输出  1101
    printf("%c", 10);    // ASCII 10 =  换行
    return 0;
}
```

为什么 13 是 1101 ？

13÷2=6 余 1 ，6÷2=3 余 0 ，3÷2=1 余 1 ，1÷2=0 余 1 ，余数倒着读

① 顺序是关键

先递归处理 n / 2（更高的位），回来再打印 n % 2（当前位）—— 低位后打，顺序才正确

② 追踪 toBin(13)

13 → 先 toBin(6) → 先 toBin(3) → 先 toBin(1) → 打印 1；回来一路打印 1、0、1，得到 1101

③ 反过来会怎样

若先打印再递归，输出变成倒着的 1011 —— 试试交换两行位置，看结果怎么变

汉诺塔：递归的巅峰入门题

```
void hanoi(int n, char a, char b, char c) {  // 
    if (n == 1) {                // 基准：只搬 1 个盘
        printf("%c -> %c", a, c);
        return;
    }
    hanoi(n - 1, a, c, b);   // n-1 个先到中转柱
    printf("%c -> %c", a, c);  // 最大盘到目标柱
    hanoi(n - 1, b, a, c);   // 再从中转到目标
}
int main(void) {
    hanoi(3, 'A', 'B', 'C');  // 2 的 3 次方减 1
    return 0;
}
```

① 规则

三根柱子，每回只能搬最上面的一个盘，大盘不许压在小盘上

② 三步走

上面 n-1 个先到中转柱 → 最大盘到目标柱 → n-1 个再从中转到目标柱

③ 移动次数 2^n - 1

3 个盘 7 步；64 个盘是天文数字 —— 汉诺塔天生指数增长

递归与 return 的配合

哈爾濱工程大學

① 有返回值的递归： return 是答案的传送带

fact 、 fib 、 gcd ：每一层 return 一份答案给上一层，上层拿它继续算 —— 答案沿原路一层层传回 main

② void 递归：靠顺序干活， return 只是刹车

toBin 、 hanoi ：不返回值，靠 " 先递归、再打印 " 的调用顺序把活干完； return 只负责提前结束本层

③ 别把答案弄丢

写了 fact(n - 1) 却忘了乘 n ，或者调用后不 return 也不存进变量 —— 答案半路蒸发，上层拿到的是垃圾

写递归前先问自己：每一层要向上交回什么？

例 1 · 递归求阶乘


```c
int fact(int n) {
    if (n == 0) {
        return 1;
    }               //  基准情形
    return n * fact(n - 1);  //  缩小一步
}
int main(void) {
    int n;
    scanf("%d", &n);
    printf("%d", fact(n));
    return 0;
}
```

题目

输入 n ，用递归函数求 n! 并输出

运行结果

输入 5 → 输出 120
输入 0 → 输出 1

要点

基准 n == 0 ，每层只乘一个数

n = 0 也能得到正确答案 1

函数只管算，打印交给 main

例 2 · 斐波那契前 n 项

```c
int fib(int n) {
    if (n <= 2) {
        return 1;
    }              //  基准：前两项都是 1
    return fib(n - 1) + fib(n - 2);
}
int main(void) {
    int i;
    for (i = 1; i <= 6; i++) {
        printf("%d ", fib(i));
    }          //  输出 1 1 2 3 5 8
    return 0;
}
```

题目

用递归函数输出斐波那契数列的前 6 项

运行结果

```
1 1 2 3 5 8
```

要点

for 负责报第几项，fib 负责算值

每项都是一次独立的递归调用

两个基准用 n <= 2 一行兜住

例 3 · 递归求最大公约数

```c
int gcd(int a, int b) {
    if (b == 0) {
        return a;
    }            //  基准：除数为 0
    return gcd(b, a % b);
}
int main(void) {
    int a, b;
    scanf("%d %d", &a, &b);
    printf("%d", gcd(a, b));
    return 0;
}
```

题目

输入两个正整数，用递归版辗转相除求最大公约数

运行结果

输入 24 36 → 输出 12

调用追踪

gcd(24,36) → gcd(36,24)

→ gcd(24,12) → gcd(12,0) → 12

a % b 让问题规模不断缩小

例 4 · 数字各位求和

```c
int sumDigits(int n) {
    if (n < 10) {
        return n;
    }           //  基准：只剩一位数
    return n % 10 + sumDigits(n / 10);
}
int main(void) {
    printf("%d", sumDigits(1234));
    printf("%d", sumDigits(9));
    return 0;
}
```

题目

用递归求一个正整数各位数字之和，

sumDigits(1234) 应得 1+2+3+4 = 10

运行结果

```
sumDigits(1234) → 10
sumDigits(9)    → 9
```

要点

n % 10 取末位，n / 10 砍末位

基准 n < 10 ，一位数自己就是和

不需要任何批量存数据的工具

例 5 · 十进制转二进制输出


```c
void toBin(int n) {
    if (n > 1) {
        toBin(n / 2);  // 先处理更高位
    }
    printf("%d", n % 2);  // 再打印当前位
}
int main(void) {
    toBin(13);
    printf("%c", 10);
    toBin(7);
    return 0;
}
```

题目

把十进制正整数按二进制输出：13 的二进制是 1101

运行结果

```
1101
111
```

要点

先递归再打印：低位最后才输出

交换这两行，输出会整个倒过来

printf("%c", 10) 输出换行符

错误盘点：递归的六种翻车

哈爾濱工程大學

✗ **忘写基准情形** —— 一路递归停不下来，栈耗尽，程序崩溃

✗ **基准不可达** —— 出口写在 `n == 1` ，输入却是 `0` ，永远够不着

✗ **递归方向不收敛** —— 写成 `fact(n + 1)` ，越递越大，离出口越来越远

✗ **返回值没接住** —— 调用后既不 `return` 也不存进变量，答案半路丢失

✗ **重复计算** —— `fib` 里同一个 `fib(3)` 被反复算多遍， `n` 一大就非常慢

✗ **以为各层共用变量** —— 其实每层有自己的 `n` 副本，互不相干（呼应值传递）

小结

递归 = 基准情形 + 向基准逼近，先展开、后回收

口诀：说清任务 → 找到出口 → 缩小一步

每层调用一份副本，答案靠 return 层层传回

下讲预告 数组 —— 把同类型的数据排成一排统一管理